

PARALLELIZED BFGS USING MPI*

LILY GRAY[†] AND CHRISTINA HYMER[‡]

Abstract. This project aims to present an MPI-based parallel implementation of the quasi-Newton *Broyden-Fletcher-Goldfarb-Shanno* (BFGS) method. We will focus on distributed gradient computation, Hessian approximations, and reducing communication overhead. The expected results will demonstrate improved scalability and convergence rates compared to serial methods. Experiments will be conducted on the University of Kansas High Performance Computing (HPC) Community Cluster.

1. Introduction. Quasi-Newton methods are some of the most widely used Newton-type methods for nonlinear optimization problems. Although based on Newton’s method, these quasi-Newton methods use a different formula to compute the search direction and use an approximation to the Hessian matrix instead of using its exact calculation, which lowers the overall cost of the algorithm. Because of this, these methods have slower convergence than Newton’s method but a lower cost-per-iteration. Generally, these methods compute the search direction s by solving the system $B_k s = -\nabla f(x_k)$, where B_k is a positive definite matrix that approximates the Hessian. Various types of quasi-Newton methods differ in their choice of B_k .

The focus of our project is the parallelization of the *Broyden-Fletcher-Goldfarb-Shanno* (BFGS) method. BFGS, named for its co-inventors, is a quasi-Newton algorithm first proposed in 1970 to solve unconstrained optimization problems, and it is one of the most widely used and effective quasi-Newton update schemes. The BFGS method is summarized in Alg. 1.1 (adapted from [3, 4]). BFGS normally has a superlinear convergence rate, and a line search can be used to enhance the effectiveness of the method. In fact, for a quadratic objective function with dimension n , an exact line search guarantees that the BFGS method terminates with the exact solution in at most n iterations.

Algorithm 1.1 Broyden-Fletcher-Goldfarb-Shanno (BFGS) Method

x_0 = initial guess

B_0 = initial Hessian approximation

for $k = 0, 1, 2, \dots$ **do**

 Solve $B_k s_k = -\nabla f(x_k)$ for s_k

 Use a line search to determine $x_{k+1} = x_k + \alpha_k p_k$, where α_k is the step length.

 Compute $y_k = \nabla f(x_{k+1}) - \nabla f(x_k)$.

 Compute

$$B_{k+1} = B_k - \frac{(B_k s_k s_k^\top B_k)}{s_k^\top B_k s_k} + \frac{y_k y_k^\top}{y_k^\top s_k}.$$

end for

The parallelization of optimization algorithms began as a way to efficiently solve medium- to high-scale optimization applications that require efficient techniques to improve accuracy and computational efficiency. For example, since neural networks

*A Project Proposal for EECS 800: Optimization II. Submitted on March 24, 2025.

[†]University of Kansas, Department of Electrical Engineering and Computer Science, Lawrence, KS (lgray@ku.edu).

[‡]University of Kansas, Bioengineering Program, Lawrence, KS (chymer@ku.edu).

in machine learning are naturally parallel in their construction, the development of parallel or distributed optimization algorithms has become a well-established strategy to increase learning rates without significantly increasing computational time or complexity [6, 9]. The convergence achievable with a given quasi-Newton method is often a trade-off between performance and computational requirements; parallelization of the method aims to increase its convergence rate and accuracy.

Our motivation for this project comes from the need for optimization in applied mathematics, engineering, medicine, economics, and other sciences, despite the increasing computational complexity and memory requirements of new algorithms and applications. Therefore, we propose research into an MPI-based parallelization of the BFGS method to hypothetically reduce computational complexity and time requirements while increasing the convergence speed.

2. Background & Literature Review. In recent years, new optimization methods have been developed and theoretical studies have been conducted, demonstrating the need for parallelization of optimization algorithms to reduce computational costs and memory requirements [6, 10]. In particular, due to their increasingly interdisciplinary nature, parallel quasi-Newton methods have been rapidly explored in several directions. Some applications of parallel quasi-Newton optimization methods include network flow problems, stochastic optimization, optimal control, discrete optimization, multiobjective programming, and approximation techniques [10].

Several studies have been conducted on parallel quasi-Newton methods, and many of these agree that parallel implementations of BFGS are among the most effective methods.

One of the first parallelized BFGS algorithms was developed in 1988 by Byrd, Schnabel, and Schultz [1]. They aimed to utilize "multiple processors to perform multiple function and derivative evaluations simultaneously," and their primary goal was to implement parallelization in BFGS "where function evaluation [was] expensive". The authors were able to cut the computational costs of the Hessian approximation, B_k , update by computing matrix-vector multiplications rather than triangular solves. Another parallelization they implemented was using different processors to compute different parts of the Hessian matrix, gradient, and function. The "dominant cost" in "many practical problems" using quasi-Newton methods comes from "function and derivative evaluation." Their results indicated that as the number of columns in the Hessian increased, the average speedup also increased.

In 1991, Charles Still proposed "a parallel formulation of the BFGS method for unconstrained minimization" that "decompos[ed] the iteration and update equations with an orthonormal basis" [8]. Still's group actually used a parallelized version of inverse BFGS, but still sent the $B_{i,j}$ update to different processors via distributed memory. As the number of processors increased by doubling from 1 to 128, the amount of time it took the algorithm to converge decreased on the parallel computing computers nCube and Symult. However, increasing the number of nodes quickly reaches a plateau in terms of convergence speed, which is a phenomenon known as Amdahl's Law.

Shortly after Still, in 2000, Seán McLoone developed a parallelized "variable memory quasi-Newton neural network training algorithm" [6]. This was a novel development because "unlike existing training methods" it was "able to optimize performance in relation to available memory." The author was able to show that variable memory was always more computationally efficient than a full memory approach, which partitioned data equally among all storage locations. In practice, the variable memory

approach partitioned data using a master-slave schema based on the available memory of the system.

In 2020, Tong et. al detailed their parallelized L-BFGS algorithm. Parallelized versions of quasi-Newton methods like L-BFGS are growing more popular as people are wanting to solve larger-scale problems with massive amounts of data. The authors claim that their parallelization was the "first one that truly parallelize[d] L-BFGS with a convergence guarantee" [9]. Previous attempts at parallelization "only [parallelize] the calculation for gradient or the two-loop recursion." Their new algorithm was able to achieve the "same linear convergence rate" of L-BFGS but with significant speedup.

3. Research Question & Methodology. This project aims to develop and test an algorithm for MPI-parallelized BFGS optimization, optimize communication overhead in distributed Hessian approximations, and evaluate performance scalability. Put simply, our research will investigate the following question: Will parallel BFGS converge faster than serial BFGS while maintaining the method's accuracy?

Much previous research in parallelized BFGS has primarily focused on L-BFGS (Limited Memory BFGS), which is more common in machine learning and not exactly the approach we wish to take with our research. We wish to develop a unique parallelization of BFGS with distributed Hessian and gradient computations and compare the parallelized algorithm to serial by applying both algorithms to relevant optimization problems.

The literature mentioned previously lacked an empirical comparison between the performance of the parallel and serial algorithms. We will fix that in our proposed research project.

Our research will proceed in two phases. During the first phase, we will create an MPI-based parallel implementation of the BFGS algorithm. This implementation will include distributed Hessian approximation and parallel gradient computation: each rank computes local gradients and aggregates distributed gradient updates via **MPI.Allreduce**. Each MPI rank thus maintains a local history of past gradients and updates the Hessian approximation accordingly. A parallel line search will be performed using **MPI.Bcast** for step size synchronization. This first phase also includes code debugging and a serial implementation of BFGS to compare with its parallelization. Both serial and parallel implementations of BFGS will utilize the Golden Section Line Search (GSLs) with Strong Wolfe stopping conditions [5, 4, 7].

In the second phase, we will apply the serial and parallel BFGS algorithms to different relevant optimization problems to compare convergence rates and complete convergence analyses.

Experiments will be conducted on the University of Kansas HPC Community Cluster. Finally, we will discuss the scalability and performance metrics of the parallel version of BFGS [2].

4. Conclusions. BFGS is regarded as one of the best quasi-Newton methods for solving unconstrained nonlinear optimization problems. However, there are steps in the algorithm that are immensely computationally complex and can become burdensome for large-scale problems. Parallelization is a technique for cutting computational/communication costs and overheads that is growing in popularity due to increasingly large amounts of data in modern optimization problems. We propose research that will yield a novel parallelization of BFGS as well as unique numerical experiments and convergence analyses comparing parallelized BFGS to serial BFGS. The benefits of completing this research project are

1. The development of a new parallel algorithm with a faster convergence than

128 BFGS that can be used to solve large problems

129 2. The convergence analysis can be used in future works as a jumping-off point
130 for other parallel BFGS algorithms

131 3. The applications of parallel and serial BFGS may inspire others to produce
132 future work towards more applications of parallelized quasi-Newton methods

REFERENCES

- [1] R. H. BYRD, R. B. SCHNABEL, AND G. A. SHULTZ, *Parallel quasi-Newton methods for unconstrained optimization*, 1988, <https://apps.dtic.mil/sti/tr/pdf/ADA193250.pdf>.
- [2] A. GRAMA, A. GUPTA, G. KARYPIS, AND V. KUMAR, *Introduction to Parallel Computing*, Addison-Wesley, 2 ed., 2003.
- [3] I. GRIVA, S. G. NASH, AND A. SOFER, *Linear and Nonlinear Optimization*, SIAM: Society for Industrial and Applied Mathematics, Philadelphia, PA, 2 ed., 2008, <https://epubs.siam.org/doi/abs/10.1137/1.9780898717730>.
- [4] M. T. HEATH, *Scientific Computing: An Introductory Survey*, McGraw-Hill, Boston, MA, 2 ed., 2002.
- [5] D. G. LUENBERGER AND Y. YE, *Linear and Nonlinear Programming*, Springer, New York, NY, 3 ed., 2008, <https://doi.org/10.1007/978-0-387-74503-9>.
- [6] S. MCLOONE, *A parallel variable memory BFGS training algorithm*, IFAC Proceedings Volumes, 33 (2000), pp. 129–134, [https://doi.org/10.1016/S1474-6670\(17\)35459-9](https://doi.org/10.1016/S1474-6670(17)35459-9). 6th IFAC Workshop on Algorithms and Architectures for Real-Time Control 2000, Palma de Mallorca, Spain.
- [7] J. NOCEDAL AND S. J. WRIGHT, *Numerical Optimization*, Springer, New York, NY, 2 ed., 2006, <https://doi.org/10.1007/978-0-387-40065-5>.
- [8] C. H. STILL, *The parallel BFGS method for unconstrained minimization*, in The Sixth Distributed Memory Computing Conference Proceedings, Los Alamitos, CA, USA, May 1991, IEEE Computer Society, pp. 347–354, <https://doi.org/10.1109/DMCC.1991.633160>.
- [9] Q. TONG, G. LIANG, X. CAI, C. ZHU, AND J. BI, *Asynchronous parallel stochastic quasi-Newton methods*, 2020, <https://arxiv.org/abs/2011.00667>.
- [10] R. ČIEGIS, D. HENTY, B. KÅGSTRÖM, AND J. ŽILINSKAS, *Parallel Scientific Computing and Optimization: Advances and Applications*, Springer, New York, NY, 1 ed., 2009, <https://doi.org/10.1007/978-0-387-09707-7>.